

Execution Control Plane

Detailed architecture and implementation specification

United States deployment baseline

AUTHORITY	PostgreSQL WorkItems, signed plans, deterministic state transitions and fenced effects
EXECUTION	Capability resolution, governed context, bounded workers and structured results
RELIABILITY	At-least-once transport with idempotent commands, leases, outbox and reconciliation
AZURE	Container Apps, Service Bus, APIM, PostgreSQL and replaceable Foundry execution

Planning baseline | Version 1.0 | 17 September 2026

Extends the foundation and KYC implementation plan. Configuration values are proposed and require target-subscription validation.

Contents

Purpose and outcome	3
1. Responsibilities and non-responsibilities	3
2. Logical architecture	4
3. Authoritative records	5
4. Component design	5
5. WorkItem model and state machine	10
6. Runtime sequences	11
7. Database implementation	12
8. Messaging implementation	13
9. Azure deployment topology	14
10. API and event contracts	16
11. Concurrency, idempotency and effect safety	17
12. Security model	18
13. Reliability and failure semantics	18
14. Observability and flight recorder	19
15. Capacity and performance	20
16. Disaster recovery	21
17. Code structure and implementation interfaces	21
18. Implementation plan	22
19. Test strategy	23
20. Operational runbooks	24
21. Architecture decisions to close before production	25
22. Definition of done	25

Purpose and outcome

The execution control plane turns a signed HarnessPlan and a business Case into durable, authorized, bounded work. It owns lifecycle and coordination. It does not perform KYC reasoning and it does not allow agents to write business truth directly.

The recommended implementation is a small set of stateless services on Azure Container Apps, with PostgreSQL as the authority for WorkItems and control-plane state, Azure Service Bus as at-least-once delivery transport, APIM as the mandatory model and tool boundary, and either Microsoft Foundry hosted agents or an isolated Container Apps executor as the replaceable cognition runtime.

This specification expands the foundation plan. It treats the two supplied PDFs as architecture inputs, not instructions. Azure settings are proposed values to validate in the target subscriptions before deployment.

1. Responsibilities and non-responsibilities

The execution control plane owns:

- admission of a case event or command into a pinned, signed HarnessPlan;
- creation and lifecycle of durable WorkItems;
- dependency evaluation and readiness;
- capability and ExecutionProfile resolution;
- policy, authority, residency, assurance and revocation checks;
- budget reservation and reconciliation;
- context assembly requests and ContextManifest validation;
- dispatch, leasing, fencing, retries, deadlines, cancellation and compensation routing;
- validation of AgentResult envelopes and conversion to proposed BusinessCommands;
- deterministic completion and assurance gates;
- human-review routing;
- outbox publication, recovery reconciliation and operational telemetry.

It does not own:

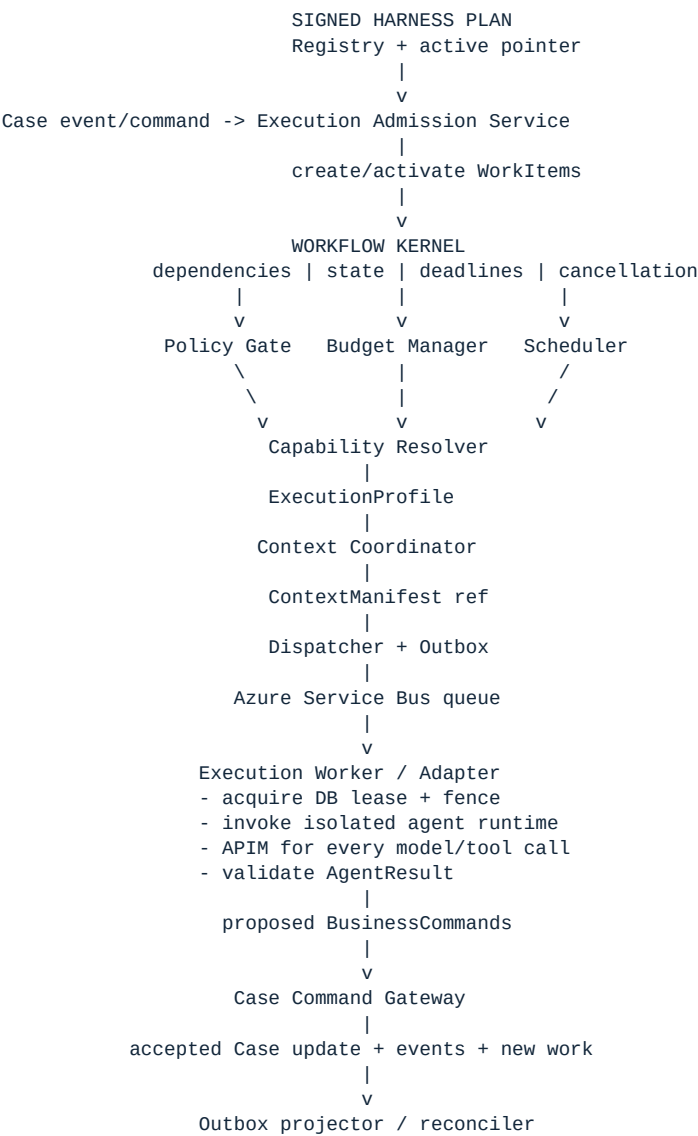
- the current Case aggregate or accepted business truth;
- immutable evidence bytes;
- final KYC policy authorship;
- model or tool credentials in agent context;
- vendor-specific endpoint knowledge inside AgentSpecs;
- analyst UI state as a source of truth;
- model conversations as durable workflow state;
- effective business decisions outside the Case Command Gateway.

The central invariant is:

agents propose -> control plane validates -> Case Service commits

An agent can return an excellent recommendation and still have no authority to make it effective. A deterministic gate can reject the recommendation because the case changed, a plan was revoked, required evidence is missing, a budget is exhausted, or human approval is required.

2. Logical architecture



Cross-cutting: Entra identity, authorization, revocation, OTel, flight recorder, audit export, rate limits, quotas and operations.

The services are logical boundaries. The first release should deploy them as five independently scalable applications, not as fifteen tiny services:

Deployment unit	Modules contained	Reason for boundary
execution-api	Admission, work query, cancellation, manual operations	Synchronous API security and latency profile
workflow-controller	Kernel, dependency evaluator, scheduler, budgets, resolver	One transactional control loop and shared database semantics
context-policy	Context coordination, authorization policy, execution policy, assurance checks	Sensitive reads, policy caching and different scale profile
execution-worker	Dispatch consumer, lease manager, runtime adapter, result validator	Untrusted workload boundary and queue-driven scale
control-reconciler	Outbox, sweepers, timers, DLQ reconciliation, recovery	Background work with elevated recovery visibility but narrow mutation rights

Split a module into another deployable only when it needs a distinct trust boundary, scaling profile, data owner or release cadence. Preserve module interfaces from the start so this does not require domain redesign later.

3. Authoritative records

The architecture avoids ambiguous ownership by assigning one authoritative record to each question.

Question	Authoritative record	Storage
What business state is effective now?	Case aggregate	PostgreSQL case_core
How did accepted business state change?	CaseEvent ledger	PostgreSQL case_ledger, signed WORM export
What work exists and what is its lifecycle?	WorkItem and WorkItemTransition	PostgreSQL workflow
What executable authority was selected?	Signed HarnessPlan and resolved ExecutionProfile	Registry plus pinned refs in PostgreSQL
What could the execution read and invoke?	PermissionEnvelope and ContextManifest	Registry/Blob plus indexed metadata
What happened computationally?	ExecutionRecord	PostgreSQL metadata plus Blob artifacts
Was a message delivered?	Service Bus delivery state	Transport only; reconstructable
What should run next?	Kernel readiness result over current WorkItems and Case refs	Recomputed and persisted transition
What did a model recommend?	AgentResult and DecisionRecommendation	Execution record and Case proposal records
What outcome became effective?	FinalDisposition	Case aggregate and ledger

Service Bus, model threads, Container Apps revisions, Search indexes and Application Insights are never authoritative for workflow or business truth.

4. Component design

4.1 Execution Admission Service

Admission accepts a committed CaseEvent, an authorized command result, a timer, or an approved operator action. It resolves the active release for the case's execution stamp, verifies the plan signature and revocation state, pins the plan and policy context, and asks the kernel to calculate new or changed WorkItems.

Input contract:

```
{
  "request_id": "req_01...",
  "tenant_id": "tenant_01...",
  "case_id": "case_01...",
  "case_sequence": 482,
  "trigger": {
    "type": "case_event",
    "ref": "evt_01..."
  },
  "requested_at": "2026-09-17T15:30:00Z"
}
```

Admission rules:

1. Authenticate the caller and derive tenant from verified identity/context.
2. Load the Case head and reject an impossible future sequence.

3. Resolve an existing case-pinned plan or the active plan for a new case.
4. Verify digest, signature, lifecycle state, revocation and geography.
5. Evaluate trigger rules with a deterministic policy engine.
6. Create WorkItems and dependency edges idempotently in one transaction.
7. Append WorkItem transition records and outbox rows in that transaction.
8. Return accepted identifiers; do not wait for agent work.

The idempotency key is `tenant:case:trigger-type:trigger-ref:plan-digest`. Repeating an identical admission returns the prior result. Reusing the key with different content is rejected.

4.2 Workflow Kernel

The kernel is the state machine and invariant engine. It is a deterministic library used by the controller and command path, not an autonomous agent. Given current Case references, WorkItems, policy output and time, it produces proposed state transitions and follow-on work.

Kernel responsibilities:

- validate legal state transitions;
- evaluate dependency predicates;
- invalidate downstream work after material Case revisions;
- enforce maximum attempts, rework cycles and graph fanout;
- calculate readiness and terminal completeness;
- route to human review or suspension;
- generate timers for deadlines and retry eligibility;
- enforce cancellation and recovery epochs;
- prevent a blocked, expired or stale WorkItem from dispatching;
- emit transition reason codes suitable for audit and operations.

The kernel never calls models, vendors or message brokers inside its transaction. It writes state and outbox intent; external effects occur after commit.

4.3 Dependency Evaluator

Dependencies are predicates over typed outputs, not simple predecessor completion flags.

```
{
  "dependency_id": "dep_01...",
  "work_item_id": "wi_decision",
  "requires": {
    "work_item_id": "wi_screening",
    "state": "COMPLETED",
    "output": "screening_complete",
    "minimum_revision": 18,
    "fresh_until_after": "2026-09-17T15:30:00Z"
  },
  "on_unsatisfied": "BLOCK",
  "on_invalidated": "REWORK"
}
```

Supported predicates include state, named output, Case or subject revision, evidence freshness, policy version, assurance result, absence of blocking contradictions, and required human action. The evaluator returns SATISFIED, UNSATISFIED, UNKNOWN or ERROR. UNKNOWN never makes work READY.

Dependency results carry an input digest. A later change to any input invalidates the result. Before finalization, the Case Command Gateway recomputes the complete decision dependency digest in the same database transaction.

4.4 Scheduler and Dispatcher

The scheduler finds READY work with no active dispatch and creates a dispatch intent. It uses `FOR UPDATE SKIP LOCKED` in short transactions so multiple scheduler replicas can compete without serializing the entire queue.

Selection order is proposed as:

```
priority class
-> regulatory/business deadline
-> oldest ready time
-> tenant fairness deficit
-> work item id tie-breaker
```

Tenant fairness prevents one large tenant from consuming every constrained model or vendor quota. The first release uses weighted deficit round robin in the scheduler, with per-tenant concurrency and rate caps stored in resolved configuration.

The scheduler does not send directly to Service Bus inside the database transaction. It inserts `dispatch_outbox` with a deterministic `dispatch_generation`. The outbox publisher sends it and records acknowledgement. A crash after send but before acknowledgement may cause a duplicate; the worker treats dispatch as a hint and re-reads authoritative `WorkItem` state.

4.5 Lease and Fencing Manager

Queue locks prevent another consumer from receiving the same delivery temporarily. They do not prevent a stale worker from calling a tool after its queue lock or database lease is lost. The control plane therefore uses an application lease and fencing epoch.

Lease acquisition atomically checks `READY` or `RETRY_SCHEDULED` state, dispatch generation, deadline, plan authority and budget availability; then it sets:

```
state           = LEASED
lease_owner     = execution_id
lease_until     = database_clock + 90 seconds
fencing_epoch   = fencing_epoch + 1
attempt_number  = attempt_number + 1
```

The worker renews at 20-second intervals when it is actively progressing. Every privileged execution action carries tenant, case, `WorkItem`, execution ID and current fencing epoch. Tool and command gateways verify the epoch against the control plane or a short-lived signed execution token derived from it. Once cancelled, expired or superseded, the epoch is invalid and new effects are denied.

Never use process time for authority decisions. Use the database clock for lease writes and persisted deadlines, and tolerate bounded clock skew only in token verification.

4.6 Capability Resolver

The resolver converts a requirement such as `screening.sanctions / HIGH` into an immutable `ExecutionProfile`. It intersects:

- AgentSpecs providing the capability;
- agent and plan lifecycle status;
- `PermissionEnvelope` and effect class;
- tenant, geography, residency and egress policy;
- model and tool availability;
- evaluation attestation and assurance class;
- dataset/vendor coverage for the subject and jurisdiction;
- current provider health and circuit state;
- quota, cost and deadline constraints;
- compatibility of agent, prompt, skill, model, tool and schema versions.

Resolver output:

```
{
  "execution_profile_id": "xp_01...",
  "profile_digest": "sha256:...",
  "plan_digest": "sha256:...",
  "agent": {"name": "screening", "version": "1.3.2"},
  "runtime": {"adapter": "foundry-hosted", "deployment": "screening-prod"},
  "model": {"profile": "reasoning.standard", "deployment": "kyc-standard-us", "version":
```

```

"pinned"},
  "tools": {"sanctions.screen": "vendor-a@3.1.0"},
  "prompt_bundle": "screening@17.0.0",
  "skill_bundle": ["entity-resolution@1.4.0"],
  "permission_envelope_ref": "perm_01...",
  "evaluation_attestation_ref": "eval_01...",
  "assurance_class": "HIGH",
  "region": "eastus2"
}

```

Health-based routing can choose only another already attested profile. Resolver fallback never silently changes geography, provider class, tool coverage or assurance.

4.7 Budget Manager

Budgets are transactional reservations, not telemetry estimates. Dimensions are:

- currency cost;
- input and output tokens;
- model calls;
- tool calls by capability/vendor;
- active runtime duration;
- investigation and rework cycles;
- concurrent executions per tenant/profile;
- daily and monthly operational caps.

Before dispatch, the controller reserves a worst-case work budget. Before each model/tool invocation, the worker obtains a sub-reservation from the budget API. When usage returns, it reconciles actual cost and releases the balance. If a provider response is ambiguous, retain the reservation until reconciliation or bounded expiry.

A reservation has HELD, CONSUMED, RELEASED or EXPIRED state and an idempotency key. PostgreSQL conditional updates prevent oversubscription. APIM rate limits and Azure quotas are additional protections but cannot replace the case budget because they do not understand Case semantics.

4.8 Context Coordinator

The execution control plane asks a separate Context Service to build the governed input; it does not concatenate Case JSON itself.

Request:

```

{
  "work_item_ref": "wi_01...@rev7",
  "execution_profile_ref": "xp_01...",
  "case_snapshot_ref": "case_01...@482",
  "required_selectors": ["subject.identity", "evidence.identity"],
  "optional_selectors": ["findings.prior_screening"],
  "token_budget": 24000,
  "purpose": "sanctions_screening"
}

```

The Context Service authenticates the control-plane identity, independently authorizes tenant/case/purpose, reads canonical data, queries derived retrieval only when allowed, applies redaction and minimization, and stores a ContextManifest. The control plane validates that all mandatory selectors resolved, the snapshot and plan match, the manifest is within size/budget, and no prohibited data class/provider combination exists.

If mandatory context cannot fit, the result is CONTEXT_TOO_LARGE or CONTEXT_INCOMPLETE. The system must not silently truncate mandatory evidence. Optional omission is recorded with reason and counts.

4.9 Policy and Assurance Coordinator

The coordinator calls a deterministic policy service at these gates:

1. admission;
2. transition to READY;

3. ExecutionProfile resolution;
4. context acceptance;
5. each tool effect;
6. AgentResult acceptance;
7. BusinessCommand submission;
8. WorkItem completion;
9. decision readiness and finalization.

Every result includes bundle digest, input digest, rules evaluated, rules fired, outcome and explanation data. Outcomes are PASS, FAIL, UNKNOWN and ERROR. Only PASS authorizes the requested transition. A rule can explicitly map UNKNOWN to HUMAN_REQUIRED, but absence of a rule never means allow.

Semantic QA is scheduled as another WorkItem. Deterministic assurance remains in the control plane and checks schemas, references, required work, freshness, contradictions, policy snapshots and disposition preconditions.

4.10 Execution Worker and runtime adapter

The execution worker consumes a dispatch, loads current state, acquires a database lease, obtains an ExecutionProfile and ContextManifest, and invokes a runtime adapter. Runtime adapters implement one interface:

```
execute(profile, request, manifest_ref, execution_token, deadline)
  -> AgentResult | typed execution error
```

Initial adapters:

- foundry-hosted: invokes the pinned hosted-agent version;
- container-apps: runs the same generic executor in the isolated executor environment;
- recorded-simulator: supplies recorded model/tool results for deterministic tests.

The adapter cannot read PostgreSQL, Blob or Search directly. It receives only the ContextManifest payload authorized for that execution. Every model and tool request goes through APIM. The control plane maintains the same contracts when the runtime changes.

The Foundry path remains conditional on the compatibility gate: demonstrate private routing, APIM mediation for required model/tool calls, dedicated identity, cancellation, deadline handling, telemetry correlation and denial of direct resource access. Microsoft documents that hosted agents run in isolated compute and have distinct networking behavior, but the exact chosen route must be proven in the target environment. [Microsoft Foundry networking](#).

4.11 Result Validator and Command Submitter

Validation order:

1. execution identity, lease and fencing epoch;
2. AgentResult contract and maximum size;
3. profile, plan, prompt, skill, model and tool references;
4. evidence existence, hashes and allowed provenance;
5. claim/finding/assumption/contradiction schemas;
6. policy and permission compliance;
7. required-output completeness;
8. cost and usage reconciliation;
9. deterministic QA applicable to this work type.

Valid proposals are translated into typed BusinessCommands. The control plane never generates database paths. The Case Command Gateway reauthenticates, reauthorizes, checks current Case/entity revisions and commits accepted changes with CaseEvents. A rejected command returns a typed result to the kernel, which chooses rework, failure, human review or recomputation.

4.12 Timer, sweeper and reconciliation services

PostgreSQL stores durable timers with `due_at`, purpose, target revision and status. The timer poller claims due rows with `SKIP LOCKED`, writes outbox signals and marks them fired idempotently. Timers cover retries, deadlines, human escalation, lease expiry, periodic review and delayed policy actions.

Sweepers identify:

- `READY` work without a live dispatch;
- expired leases;
- outbox rows without broker acknowledgement;
- execution records missing a terminal `WorkItem` transition;
- budget reservations past expiry;
- Blob staging objects not registered as evidence;
- DLQ messages requiring classification;
- cases pinned to revoked dependencies;
- `WorkItems` stuck beyond operational thresholds.

Sweepers emit repair commands through the same domain APIs. They do not update arbitrary state with ad hoc SQL.

5. WorkItem model and state machine

Minimum `WorkItem` fields:

Group	Fields
Identity	<code>tenant_id</code> , <code>case_id</code> , <code>work_item_id</code> , <code>type</code> , <code>capability</code> , <code>assurance_class</code>
Versioning	<code>revision</code> , <code>case_sequence_basis</code> , <code>plan_digest</code> , <code>policy_context_ref</code> , <code>schema_version</code>
Objective	<code>objective</code> , <code>subject_refs</code> , <code>required_outputs</code> , <code>constraints</code>
Dependencies	dependency set, <code>dependency_digest</code> , invalidation policy
Lifecycle	<code>state</code> , <code>reason_code</code> , <code>priority</code> , <code>ready_at</code> , <code>deadline_at</code> , <code>terminal_at</code>
Dispatch	<code>dispatch_generation</code> , <code>queue</code> , <code>published_at</code> , <code>broker_message_id</code>
Lease	<code>execution_id</code> , <code>lease_owner</code> , <code>lease_until</code> , <code>fencing_epoch</code> , <code>attempt_number</code>
Limits	<code>budget_definition</code> , reservation refs, <code>max_attempts</code> , <code>max_rework_cycles</code>
Result	<code>AgentResult</code> ref, accepted command refs, completion outputs, failure detail
Audit	<code>created_by</code> , transition sequence, timestamps, trace and execution refs

State transitions:

PROPOSED	
-> <code>BLOCKED</code>	dependencies or policy not satisfied
-> <code>READY</code>	all deterministic readiness gates pass
BLOCKED	
-> <code>READY</code>	dependency becomes satisfied
-> <code>CANCELLED</code>	parent/case/policy cancels need
-> <code>EXPIRED</code>	deadline passes with no permitted continuation
READY	
-> <code>LEASED</code>	worker acquires DB lease and budget
-> <code>CANCELLED</code> / <code>EXPIRED</code>	authority changes before acquisition
LEASED	
-> <code>RUNNING</code>	runtime invocation accepted
-> <code>READY</code>	lease expires before side effect; safe redispatch
-> <code>RETRY_SCHEDULED</code>	typed transient failure

-> HUMAN_REQUIRED policy or ambiguity requires a person
 -> FAILED nonretryable error or attempt limit

RUNNING

-> COMPLETED required outputs accepted by command/gates
 -> PARTIAL useful accepted output, requirements still incomplete
 -> RETRY_SCHEDULED permitted transient failure
 -> REWORK_REQUIRED output rejected or dependency changed
 -> HUMAN_REQUIRED judgment or exception path
 -> CANCELLED cancellation acknowledged; late effects fenced
 -> FAILED terminal technical/policy failure

PARTIAL / REWORK_REQUIRED

-> BLOCKED / READY kernel creates or activates remaining work

HUMAN_REQUIRED

-> WAITING_HUMAN review task created and assigned
 -> READY / COMPLETED authorized human action accepted
 -> SUSPENDED operational pause

Any nonterminal state

-> CANCELLED / EXPIRED when policy permits

Store every transition in `work_item_transitions` with from/to, revision, reason, actor, input digest and time. Enforce allowed transitions in one shared kernel package and database constraints. Do not let each worker implement its own state machine.

6. Runtime sequences

6.1 Normal agent execution

1. Scheduler writes `dispatch_outbox` for READY WorkItem generation 4.
2. Publisher sends `MessageId` `tenant:work:4` and records broker acceptance.
3. Worker receives with `PeekLock` and reads WorkItem.
4. Worker atomically acquires application lease, epoch 9, and budget reservation.
5. Resolver returns a pinned, attested `ExecutionProfile`.
6. Context Service creates `ContextManifest` for Case sequence 482.
7. Worker starts `ExecutionRecord` and invokes runtime adapter.
8. Agent calls `model/tools` only through APIM with scoped execution token.
9. Worker validates `AgentResult` and submits typed `BusinessCommands`.
10. Case Gateway commits accepted changes, `CaseEvents` and new outbox rows.
11. Kernel verifies required outputs and marks WorkItem COMPLETED.
12. Worker closes `ExecutionRecord`, releases unused budget and completes message.
13. `CaseEvent` outbox wakes the kernel for dependent WorkItems.

The Case commit occurs before the Service Bus message is completed. If the worker crashes after step 10, redelivery finds the stored command receipt and terminal WorkItem result rather than duplicating the business effect.

6.2 Tool call with evidence

Agent -> APIM: ToolRequest + execution token + semantic operation key
 APIM -> Tool Gateway: verified caller, fixed capability route
 Tool Gateway -> Control Plane: validate epoch, permission, budget and policy
 Tool Gateway -> Vendor: scoped credential; provider idempotency when supported
 Vendor -> Tool Gateway: raw response or typed failure
 Tool Gateway -> Blob staging: raw bytes
 Evidence service: malware/content checks, exact hash, immutable version, read-back
 Evidence service -> Case Gateway: RegisterEvidence command
 Case Gateway: evidence metadata + CaseEvent + outbox commit
 Tool Gateway -> Agent: ToolResult with evidence ref, status and bounded content

SUCCESS_EMPTY means a successful covered query returned no candidates. TIMEOUT, RATE_LIMITED, AUTH_FAILURE, PROVIDER_ERROR and INVALID_RESPONSE never become empty evidence. If the provider may have committed an irreversible effect but the response is unknown, return RECONCILIATION_REQUIRED and prohibit blind retry.

6.3 Lease loss during execution

worker heartbeat fails
 -> database lease expires
 -> sweeper transitions WorkItem to READY and increments dispatch generation
 -> new worker acquires epoch 10
 -> old worker tries tool/command with epoch 9
 -> gateway rejects STALE_FENCE
 -> old execution closes as superseded when connectivity returns

If epoch 9 already completed an external action, its operation receipt and provider identifier are reconciled. Epoch 10 reuses the semantic operation key and first queries the receipt/provider instead of repeating the action.

6.4 Cancellation

Cancellation is a durable command:

1. authorize the requester and reason;
2. increment case/work authority epoch;
3. mark eligible WorkItems CANCELLED and revoke active execution tokens;
4. publish cancellation signals after commit;
5. deny new model/tool/command calls for old epochs;
6. wait for workers to acknowledge or leases to expire;
7. reconcile effects already accepted externally;
8. schedule compensation only through an authorized compensating command.

Cancellation cannot make an already accepted external effect disappear. The audit record distinguishes requested, effective and reconciled cancellation.

6.5 Human review

The kernel creates a HumanReview WorkItem with the same dependency, snapshot and deadline model. The analyst UI loads a review packet through the Context Service. The submitted action includes Case sequence and dependency digest. The Case Gateway rejects stale actions after evidence, policy or identity changes. Four-eyes rules are authorization policy, not UI convention.

7. Database implementation

Use a dedicated workflow schema on the Case PostgreSQL server for transactional proximity to Case state. The workflow module cannot update Case tables directly; the Case Service owns those procedures. The controller uses Entra-authenticated roles and PgBouncer transaction pooling on port 6432 after driver validation. Microsoft documents built-in PgBouncer support, including Entra authentication. [Microsoft: PgBouncer](#).

Core tables:

```
workflow.work_items
workflow.work_item_dependencies
workflow.work_item_transitions
workflow.dispatch_outbox
workflow.inbox_receipts
workflow.execution_leases
workflow.budget_accounts
workflow.budget_reservations
workflow.durable_timers
workflow.provider_operation_receipts
workflow.reconciliation_cases
workflow.dead_letter_cases
execution.execution_records
execution.execution_artifacts
assurance.policy_results
assurance.assurance_results
```

Important constraints:

- composite tenant and case foreign keys on every child table;
- unique (tenant_id, work_item_id) and immutable ID generation;
- unique (tenant_id, semantic_idempotency_key) for command/effect receipts;
- unique (tenant_id, work_item_id, dispatch_generation) for dispatch;
- unique (tenant_id, work_item_id, transition_sequence) for history;
- check constraints for legal timestamps, positive budgets and valid enum values;

- FORCE ROW LEVEL SECURITY on tenant-scoped tables;
- runtime roles are nonowner and cannot BYPASSRLS;
- indexes begin with tenant_id and match scheduler/query order;
- append-only transition and result artifacts for runtime roles;
- database functions use fixed search_path and least privilege.

Lease acquisition pseudocode:

```
BEGIN;
SELECT state, revision, dispatch_generation, fencing_epoch
FROM workflow.work_items
WHERE tenant_id = :tenant
  AND work_item_id = :work
FOR UPDATE;

-- Application verifies state, generation, deadline, revocation and budget.
UPDATE workflow.work_items
SET state = 'LEASED',
    revision = revision + 1,
    execution_id = :execution,
    lease_until = transaction_timestamp() + interval '90 seconds',
    fencing_epoch = fencing_epoch + 1,
    attempt_number = attempt_number + 1
WHERE tenant_id = :tenant
  AND work_item_id = :work
  AND state IN ('READY', 'RETRY_SCHEDULED')
  AND dispatch_generation = :generation;

INSERT INTO workflow.work_item_transitions (...);
INSERT INTO execution.execution_records (...);
COMMIT;
```

The real implementation uses a stored command function or repository transaction with strict row-count checks. It obtains no model/tool data while the row is locked.

Scheduler query approach:

```
SELECT tenant_id, work_item_id
FROM workflow.work_items
WHERE state = 'READY'
  AND ready_at <= transaction_timestamp()
  AND deadline_at > transaction_timestamp()
  AND no_live_dispatch = true
ORDER BY priority_rank, deadline_at, ready_at, work_item_id
FOR UPDATE SKIP LOCKED
LIMIT :batch;
```

Update selected rows and insert outbox entries in the same transaction. The actual fairness algorithm should preselect eligible tenant buckets so a global priority query cannot starve small tenants.

8. Messaging implementation

Use Azure Service Bus Premium with private access and Entra roles. Use PeekLock; ReceiveAndDelete can lose work when a consumer fails. Microsoft documents PeekLock redelivery and DLQ behavior. [Microsoft: locks and settlement](#).

Initial queue topology:

Queue	Purpose	Consumers
work.ready.screening	Screening capabilities	Screening workers
work.ready.search	Internal/external search	Search workers with distinct runtime profiles
work.ready.investigation	Long reasoning loops	Investigation workers
work.ready.decision	Recommendation work	Decision workers
work.ready.qa	Independent semantic QA	QA workers

Queue	Purpose	Consumers
work.ready.platform	Context, migration, reevaluation and internal tasks	Trusted platform workers
work.cancel	Best-effort prompt cancellation signal	All runtime adapters
work.human-notifications	Analyst notification hints	Notification adapter

Queue settings baseline:

```
sku: Premium
receiveMode: PeekLock
lockDuration: PT1M
maxDeliveryCount: 5
defaultMessageTimeToLive: P1D
deadLetteringOnMessageExpiration: true
requiresDuplicateDetection: true
duplicateDetectionHistoryTimeWindow: PT10M
requiresSession: false
localAuthentication: disabled
publicNetworkAccess: disabled
```

One-minute broker locks are separate from 90-second application leases. Auto-renew the broker lock only for bounded setup/commit time, not for an entire unbounded investigation. Long execution authority comes from the database lease; if broker settlement fails after successful work, redelivery is safe.

Duplicate detection uses application-controlled MessageId within a bounded window and can affect throughput as the window increases. Keep the 10-minute baseline and retain application idempotency indefinitely for the relevant business retention. [Microsoft: duplicate detection](#).

Do not enable sessions merely to obtain correctness. Case mutations are ordered by database transaction and preconditions. Add a separate session-enabled queue only for a capability with a proven per-case FIFO requirement; set SessionId to the tenant-scoped case key and accept reduced concurrency for a busy case. Microsoft describes sessions as an implementation of per-group sequential processing. [Microsoft: sequential convoy pattern](#).

DLQ handling:

1. alert on any persistent entry and on increasing oldest age;
2. copy broker metadata and reason into dead_letter_cases;
3. classify contract, poison data, authorization, expired work, repeated transient or platform defect;
4. fix root cause or create an approved exception;
5. create a new dispatch generation through the controller;
6. settle the DLQ message only after the durable recovery record is committed.

Never replay a DLQ directly to the original queue without rechecking current WorkItem authority.

9. Azure deployment topology

The proposed production stamp uses East US 2 as primary and Central US as recovery, pending subscription, SKU, model and policy validation.

Resource	Execution-control-plane configuration
Container Apps platform environment	Internal workload-profile environment; delegated /23; zone redundancy; execution API/controller/context/reconciler; critical minimum 3 replicas
Container Apps executor fallback	Separate internal workload-profile environment and /23; no direct data roles; 0-50 replicas; controlled egress only to APIM and platform dependencies
Microsoft Foundry	Separate account/project per environment, residency and isolation class; private endpoint; BYO VNet or approved managed network; hosted path only after compatibility proof

Resource	Execution-control-plane configuration
APIM	Premium classic internal VNet baseline, 2 units across supported zones; model and tool APIs; Entra validation; managed identity to backends; no KYC response caching
PostgreSQL	Flexible Server General Purpose candidate; zone-redundant HA; Entra-only; private delegated networking; PgBouncer candidate; 35-day PITR
Service Bus	Premium; private endpoint; 1 messaging unit initial; local auth disabled; queue-driven scale
App Configuration	Active plan pointers, circuit and feature flags; private endpoint; ETag updates; never stores secrets or full business policy
Key Vault	Signing and exceptional connector secrets; RBAC, private endpoint, purge protection; agents have no secret read role
Blob Storage	Execution records and context artifacts separate from evidence; private access, versions/immutability by record class
Monitor	Workspace-based Application Insights; OTel; PII-free operational telemetry; durable records unsampled elsewhere

Networking:

```

trusted platform subnet
  execution-api/controller/context/reconciler
    | private TLS
    +-> PostgreSQL / Service Bus / Blob / App Configuration / Key Vault
    +-> APIM internal endpoint

executor subnet or Foundry agent subnet
  agent execution only
    +-> APIM internal endpoint
    +-> required identity/runtime control endpoints
    X no PostgreSQL, Blob, Search, Key Vault or vendor data roles

connector subnet
  typed adapters
    +-> APIM/backend private routes
    +-> approved vendor FQDNs through Azure Firewall

```

For Foundry BYO VNet, reserve a dedicated delegated subnet and private endpoint subnet before resource creation. Microsoft states that network injection is chosen at creation and cannot simply be added or changed later; subnet sizing must include concurrent hosted sessions and overlapping rollout revisions. [Microsoft: Foundry networking options](#).

9.1 Container Apps scale rules

Controller and API scale on HTTP concurrency/CPU with minimum three production replicas. Workers scale by Service Bus active-message count with a user-assigned managed identity. Microsoft documents managed-identity support for Azure-resource scale rules. [Microsoft: Container Apps scaling](#).

Proposed worker rule per queue:

```

pollingIntervalSeconds: 10
cooldownPeriodSeconds: 300
minReplicas: 0
maxReplicas: 20
targetActiveMessagesPerReplica: 5
maxConcurrentExecutionsPerReplica: 2

```

Use separate worker apps for screening, search, investigation, decision and QA so quotas, CPU/memory, network permissions and rollout can differ. A global semaphore in PostgreSQL/Budget Manager caps model/vendor concurrency even if KEDA scales more replicas than downstream capacity.

9.2 Managed identities and roles

Identity	Required access	Explicit denial by absence of role
mi-hf-execution-api	Work API procedures; plan read; create cancellation/operator commands	Case table DML, Blob data, model/tool invoke
mi-hf-controller	Workflow procedures; budget/resolver metadata; dispatch outbox	Evidence bytes, vendor secrets, effective disposition
mi-hf-context-policy	Authorized Case read APIs, selected Blob/Search reads, policy bundles	Case writes, vendor invoke, release signing
mi-hf-worker-{class}	APIM invoke and narrow workflow lease/result procedures	Direct Case DB tables, Blob/Search/Key Vault, model resource roles
mi-hf-reconciler	Sweep/recovery procedures, queue send/receive, operational metadata	Arbitrary business mutation and finalization
mi-hf-apim-model	Exact model inference role	Case/evidence data roles
mi-hf-apim-tools	Adapter invocation	Direct broad vendor/customer-system permissions

Each worker class also receives an application role identifying permitted capability families. Azure RBAC protects resources; the signed PermissionEnvelope and application policy enforce tenant, case, field, purpose and effect restrictions.

10. API and event contracts

Synchronous APIs:

Endpoint	Purpose	Idempotency / concurrency
POST /execution-admissions	Admit committed trigger	Required request key and content digest
GET /cases/{caseId}/work-items	Authorized work view	Read snapshot sequence returned
POST /work-items/{id}:cancel	Request cancellation	Reason + expected revision
POST /work-items/{id}:lease	Acquire application lease	Dispatch generation and expected revision
POST /work-items/{id}:heartbeat	Renew lease	Execution ID + fencing epoch
POST /work-items/{id}:result	Submit AgentResult ref	Execution ID + fencing epoch + result digest
POST /budgets:reserve	Hold work/effect budget	Semantic reservation key
POST /budgets/{id}:reconcile	Consume/release against actual use	Usage receipt digest
POST /profiles:resolve	Resolve immutable ExecutionProfile	Work/plan/policy digest
POST /contexts	Request ContextManifest	Work/profile/snapshot digest
POST /policy:evaluate	Deterministic policy decision	Bundle/input digest
POST /reconciliation-cases	Open unknown-effect recovery	Operation key unique

All write APIs require Idempotency-Key, If-Match or equivalent expected revision where applicable, correlation ID, tenant-scoped identity and a bounded request size. Error responses use stable codes such as STALE_REVISION, STALE_FENCE, PLAN_REVOKED, DEPENDENCY_UNKNOWN, BUDGET_EXHAUSTED, POLICY_BLOCKED, CONTEXT_INCOMPLETE and RECONCILIATION_REQUIRED.

Dispatch message:

```
{
  "schema": "workflow.dispatch.v1",
  "message_id": "tenant_01:wi_01:4",
}
```

```
"tenant_id": "tenant_01",
"case_id": "case_01",
"work_item_id": "wi_01",
"dispatch_generation": 4,
"capability_family": "screening",
"priority": "P1",
"deadline_at": "2026-09-17T15:35:00Z",
"traceparent": "00-..."
}
```

Do not include PII, evidence, prompts, model configuration, secrets or trusted policy values in queue messages. The consumer rehydrates everything under current authority.

11. Concurrency, idempotency and effect safety

The system promises at-least-once delivery and effectively-once accepted business effects through receipts and transactional boundaries. It does not claim distributed exactly-once execution.

Idempotency scopes:

Boundary	Key
Admission	tenant + case + trigger ref + plan digest
Dispatch	tenant + WorkItem + dispatch generation
Execution attempt	WorkItem + attempt number + fencing epoch
BusinessCommand	tenant + case + WorkItem + semantic operation + logical target
Read-only tool query	tenant + case + WorkItem + query purpose + normalized input digest + dataset/version expectation
External write	tenant + business operation + logical external target; provider key when supported
Outbox publication	event/dispatch ID + destination
Inbox consumption	consumer + message/event ID

An idempotency record stores the canonical request digest. A repeated key with different content is a conflict, not a retry.

Concurrency strategy:

- use short row locks only for transition/receipt/commit boundaries;
- use optimistic entity revisions for agent proposals;
- allow commutative append of independent evidence after validation;
- invalidate dependent screening/decisions after identity or ownership changes;
- recompute final dependencies in the Case transaction;
- never hold a database transaction across network calls;
- partition scheduling and indexes by tenant/stamp before considering database sharding;
- use fencing for late workers and recovery-region writers.

Effect classes:

Effect	Retry behavior
READ_ONLY	Bounded retry for typed transient failures; persist successful evidence once
IDEMPOTENT_WRITE	Retry with stable provider/application key and receipt lookup
REVERSIBLE_WRITE	Retry only with stable key; define explicit authorized compensation
IRREVERSIBLE_WRITE	No blind retry; query provider state or human reconcile on ambiguity
EXTERNAL_COMMUNICATION	Stable message intent, dedupe and approval; delivery result is separate from content approval

Effect	Retry behavior
FINANCIAL_OR_LEGAL_EFFECT	Strong human/policy authorization, receipt and reconciliation path

12. Security model

Trust zones:

1. trusted deterministic platform: controller, Case/Command, context/policy and reconciler;
2. bounded cognition: Foundry hosted agent or isolated executor;
3. controlled capability adapters: tool services holding narrow credentials;
4. enterprise and external systems;
5. human analyst and administrator channels.

The agent runtime is treated as untrusted compute even when the organization owns the code. It receives no database credential, storage role, signing key, vendor secret or broad OAuth token.

Execution token claims:

```
{
  "iss": "harness-execution-authority",
  "aud": "harness-capability-gateway",
  "sub": "agent:screening",
  "tenant_id": "tenant_01",
  "case_id": "case_01",
  "work_item_id": "wi_01",
  "execution_id": "exec_01",
  "plan_hash": "sha256:...",
  "permission_hash": "sha256:...",
  "fencing_epoch": 9,
  "capabilities": ["model.reasoning.standard", "sanctions.screen"],
  "exp": 1789668720
}
```

Maximum token lifetime is proposed at two minutes. Renewal requires a live lease. APIM validates Entra identity; the backend validates the execution token and current authority. APIM deletes caller-supplied tenant/principal headers and sets verified context. Never authorize a tenant from an arbitrary header or request body.

Security tests include direct calls from the executor to PostgreSQL/Blob/Search/model endpoints, forged tenant/case/token claims, replay after lease loss, cross-tenant identifiers, SSRF through a tool request, prompt injection attempting to grant capabilities, plan revocation during execution, and leakage through telemetry/error bodies.

13. Reliability and failure semantics

Standard execution outcomes:

```
COMPLETED
PARTIAL
BLOCKED
RETRYABLE_ERROR
NON_RETRYABLE_ERROR
POLICY_BLOCKED
HUMAN_REQUIRED
CANCELLED
DEADLINE_EXCEEDED
BUDGET_EXHAUSTED
SUPERSEDED
RECONCILIATION_REQUIRED
```

Failure mapping:

Failure	Control-plane response
PostgreSQL unavailable before lease	Abandon message; no authority acquired; retry by broker/client policy

Failure	Control-plane response
PostgreSQL unavailable after external effect	Do not repeat blindly; persist/recover provider receipt and open reconciliation
Service Bus unavailable after DB commit	Outbox remains pending; publisher retries; sweeper monitors age
Worker crash before effect	Lease expires; redispach new generation/epoch
Worker crash after command commit	Redelivery finds command receipt and terminal result
APIM/model rate limit	Release/retain budget as appropriate; scheduled backoff within deadline
Tool timeout	Typed failure; bounded retry only if effect class permits
Context Service incomplete	Block execution; create remediation/rework, never silently omit mandatory input
Plan/model/tool revoked	Stop new dispatches; fence affected calls; identify in-flight and historical impact
Policy service unavailable	Fail closed for transitions requiring authorization; existing safe waits continue
DLQ entry	Create durable operational case; reauthorize before new dispatch
Recovery-region activation	Increment stamp authority epoch and fence primary before enabling writes

Circuit breakers exist per provider/profile/tenant where appropriate. They protect shared dependencies, but opening a circuit is not evidence of a negative KYC result. The resolver can choose only an approved attested alternative or route work to waiting/human handling.

14. Observability and flight recorder

OTel hierarchy:

```
harness.run
  workflow.admission
  workflow.transition
  profile.resolve
  budget.reserve
  context.assemble
  work.dispatch
  work.lease
  agent.execution
    model.inference
    tool.execution
    result.validate
  case.command
  assurance.check
  review.route
```

Stable operational fields include service, region, environment, tenant pseudonym, case/work/execution/trace IDs, plan/profile/policy digests, capability, state transition, outcome, error type, attempt, queue age, lease age, tokens, cost and latency. Never place names, birth dates, documents, evidence content, prompts, responses, access tokens or secrets in span attributes.

Durable ExecutionRecord contains the exact resolved dependency versions, ContextManifest, model/tool calls, AgentResult, proposed and accepted commands, policy results, usage, timing and terminal reason. Large permitted artifacts live in Blob by exact version/hash. It records observable inputs and outputs, not private model chain-of-thought.

Dashboards:

- WorkItems by state, capability, priority and age;
- READY-to-lease and end-to-end latency percentiles;
- queue active/dead-letter/oldest age and publish lag;
- lease acquisition, expiry, heartbeat and stale-fence rates;
- retry, rework, partial, human and reconciliation rates;

- model/tool error, throttle, cost and token usage;
- budget held/consumed/expired and reconciliation lag;
- policy outcomes and revocation freshness;
- controller/database saturation and scheduler batch duration;
- tenant fairness and quota consumption.

Initial alerts: any unauthorized finalization or cross-tenant access; outbox age over 60 seconds; READY or queue age over 120 seconds; any persistent DLQ; lease-expiry surge; stale revocation cache; budget anomaly; reconciliation case past SLA; database CPU above 70% for 15 minutes; storage above 80%; provider error rate above 5% for 5 minutes. Tune with preproduction data.

15. Capacity and performance

Planning load from the foundation document is 1,000 cases/day, 12 executions/case, 10x hourly burst and about 28,000 maximum tokens/execution. This yields 12,000 executions/day. At 30 seconds mean active time, average concurrency is about 4.2 and the 10x planning burst is about 42.

Initial limits:

Control	Value	Purpose
Worker concurrency per replica	2	Bound memory and concurrent model/tool use
Worker max replicas across queues	50 per stamp	Match 10x planning concurrency with headroom
Lease	90 seconds, heartbeat 20 seconds	Fast recovery with enough heartbeat tolerance
Scheduler batch	100 rows every 1 second when backlog exists	Avoid long locks; tune from DB metrics
Outbox batch	100 rows, 10 concurrent sends	Bound in-flight memory and broker calls
Context cap	24k input tokens initial	Force selection/minimization
Output cap	4k tokens initial	Bound latency/cost
Case budget	\$20, 200 tools, 30 active machine minutes	Source-derived planning guardrail, pending validation
Max rework cycles	2 initial	Stop loops and route to human

Autoscaling is limited by the smallest downstream capacity: model TPM/RPM, vendor requests, database connections, connector throughput, subnet IPs or approved case budget. Worker scaling must not outrun those limits.

Load tests:

1. sustain expected hourly peak for two hours;
2. run 10x burst for one hour, then measure drain time;
3. inject 5% long investigations and 2% vendor timeouts;
4. force controller replica turnover and database HA failover;
5. create duplicate dispatches and lost broker acknowledgements;
6. verify tenant fairness under one noisy tenant;
7. test Foundry revision rollout IP headroom;
8. verify no unsafe bypass when quota/budget is exhausted.

16. Disaster recovery

The database is the recovery anchor. Service Bus can be rebuilt from WorkItems and outbox state. Signed plan and policy artifacts must exist in the recovery region before production release.

Recovery procedure:

1. declare incident and stop new ingress/dispatch;
2. increment the global stamp authority epoch in an independently available authority;
3. fence or disable primary APIM/runtime identities and confirm no primary writers;
4. determine PostgreSQL recovery point and promote/restore recovery database;
5. validate signed registry, plan, policy and revocation data;
6. restore private DNS/routes/identities for the recovery stamp;
7. verify Case/WorkItem/ledger sequence and evidence durability watermarks;
8. expire all old leases and create new dispatch generations;
9. rebuild READY and RETRY_SCHEDULED messages from database state;
10. reconcile external operations spanning the lost interval;
11. run synthetic admission, execution, command and audit tests;
12. reopen limited analyst traffic, then controlled automation.

Never enable the recovery writer while the old stamp might still accept writes. An async database replica can lack a command whose external provider effect succeeded; provider operation reconciliation is mandatory before replay.

Proposed targets are zone-event RPO 0 for committed PostgreSQL under synchronous HA, regional RPO at most five minutes and RTO at most four hours. These remain targets until timed drills demonstrate them. Recovery-region writes are disabled during normal operation.

17. Code structure and implementation interfaces

```
runtime/execution-control/
  api/                admission, work query, cancellation
  kernel/             state machine, dependencies, invalidation
  scheduler/          fairness, eligibility, dispatch planning
  leases/             acquisition, heartbeat, fencing
  budgets/            accounts, reservations, reconciliation
  resolver/           capabilities and ExecutionProfile
  coordinator/        context, policy and assurance calls
  dispatch/           outbox publisher and queue contracts
  worker/             consumer and execution lifecycle
  adapters/
    foundry/
      container_apps/
        recorded_simulator/
  results/            AgentResult validation and command mapping
  timers/             durable timer poller
  reconciliation/      sweepers, DLQ and unknown effects
  contracts/          OpenAPI/JSON Schema/event schemas
  persistence/        repositories and migrations
  telemetry/          OTel conventions and recorder
```

Key interfaces:

```
WorkflowKernel.evaluate(snapshot) -> TransitionPlan
DependencyEvaluator.evaluate(work, inputs) -> DependencyResult[]
Scheduler.select(now, capacity) -> DispatchIntent[]
LeaseService.acquire(work, generation, execution) -> Lease
LeaseService.renew(lease, epoch) -> Lease
CapabilityResolver.resolve(requirement, authority, health) -> ExecutionProfile
BudgetService.reserve(scope, maximum, key) -> Reservation
ContextCoordinator.assemble(request) -> ContextManifestRef
PolicyCoordinator.evaluate(gate, refs) -> PolicyResult
RuntimeAdapter.execute(envelope) -> AgentResult
```

```
ResultValidator.validate(result, authority) -> ValidatedProposals
CommandSubmitter.submit(commands, lease) -> CommandReceipt[]
Reconciler.scan(category, watermark) -> RepairProposal[]
```

Use one canonical serialization library for digests and idempotency hashes. Every interface returns machine-readable reason codes and correlation references. No interface returns a raw exception as a business result.

18. Implementation plan

Phase	Duration	Work	Exit evidence
E0 - Decisions and prototypes	Weeks 1-2	ADRs, WorkItem/state contracts, PostgreSQL locking prototype, Service Bus semantics, Foundry/APIM compatibility spike	Runtime path chosen; state/authority invariants approved
E1 - Persistence and kernel	Weeks 3-5	Workflow schema, transitions, dependencies, admission, timers, RLS, idempotency	Deterministic lifecycle tests; duplicate/state transition denial
E2 - Dispatch and leases	Weeks 5-7	Scheduler fairness, outbox, Service Bus, lease/heartbeat/fencing, cancellation	Crash/redelivery/state-worker tests pass
E3 - Resolution and budgets	Weeks 6-8	Capability resolver, attestations, health, budget reservations, circuit state	Only qualified profiles resolve; concurrency/cost caps enforced
E4 - Context/policy integration	Weeks 7-9	ContextManifest orchestration, deterministic gates, assurance results	Missing/oversized/unauthorized context blocked
E5 - Runtime and results	Weeks 8-10	Foundry and Container Apps adapters, execution token, APIM calls, result validation, command submission	End-to-end synthetic WorkItem completes with no direct data access
E6 - Reconciliation and operations	Weeks 10-12	Sweepers, DLQ, unknown-effect workflow, dashboards, alerts, audit/flight recorder	Failure matrix and operational runbooks exercised
E7 - Scale and recovery	Weeks 12-14	Load, HA, regional recovery, fairness, quota and cost tests	Timed recovery and 10x burst evidence
E8 - KYC integration	Weeks 14-16	Bind all eight KYC AgentSpecs/work types and human review	Full KYC golden cases traverse control plane

Detailed engineering backlog:

ID	Deliverable	Acceptance criterion
EC01	WorkItem, transition and dependency schemas	Compatibility fixtures and illegal-transition tests
EC02	Admission API and plan pinning	Repeated trigger returns same WorkItems; revoked plan rejected
EC03	Kernel library	Same snapshot produces same TransitionPlan and digest
EC04	PostgreSQL roles/RLS/migrations	Actual runtime identities fail cross-tenant and owner-bypass tests
EC05	Transactional dispatch outbox	Crash before/after send produces no lost authoritative work

ID	Deliverable	Acceptance criterion
EC06	Scheduler/fairness	Small tenant meets latency target under noisy-tenant load
EC07	Service Bus queues and clients	PeekLock, retry, DLQ and duplicate scenarios verified
EC08	Lease/fencing/token authority	Stale worker cannot call tool or submit command
EC09	Durable timers	Retry/deadline events fire once effectively under replica turnover
EC10	Capability resolver	Residency, revocation, coverage and attestation exclusions tested
EC11	Budget accounts/reservations	Parallel reservations cannot overspend; ambiguous usage reconciles
EC12	Context coordinator	Mandatory omission blocks; trust/redaction manifest is auditable
EC13	Policy/assurance coordinator	UNKNOWN/ERROR cannot authorize transition
EC14	Foundry adapter	Private APIM-mediated path and identity proof passes
EC15	Container Apps adapter	Same contracts and denial tests pass as fallback
EC16	Result validator/command mapper	Malformed, stale, unsupported and evidence-free findings rejected
EC17	Flight recorder	Every terminal execution reconstructs versions, context and outcomes
EC18	Sweepers/DLQ/reconciliation	Repairs use typed commands; unknown effects are not blindly repeated
EC19	OTel dashboards/alerts	Failure injection creates owned, actionable alert without PII
EC20	Load/HA/DR	Peak, failover and regional drill meet approved targets

19. Test strategy

Unit and property tests:

- every legal and illegal state transition;
- dependency truth tables including UNKNOWN and revision invalidation;
- canonical digest stability;
- retry classification and backoff bounds;
- budget arithmetic, parallel reservations and release;
- capability resolution exclusion reasons;
- idempotency key/content digest conflicts;
- schedule fairness and deadline ordering.

Integration tests:

- PostgreSQL transaction plus outbox crash points;
- broker send acknowledgement loss;
- message redelivery before and after command commit;
- application lease versus broker lock loss;
- plan revocation during each execution phase;
- APIM identity/token propagation and header stripping;
- ContextManifest authorization and Search-filter bypass attempts;
- evidence durability before command acceptance;
- Entra token refresh through PgBouncer and HA failover;

- Foundry and Container Apps adapter contract parity.

Fault and adversarial tests:

- worker kill at every lifecycle boundary;
- database failover during lease, result and command phases;
- model/vendor throttle, timeout, malformed and delayed responses;
- poison queue message and oversized AgentResult;
- injected instructions requesting tools, secrets or approval;
- forged tenant/case/work/epoch values;
- cancellation racing with tool and command submission;
- recovery with an ambiguous external side effect;
- duplicate human action against a changed Case snapshot.

Release gates:

1. zero cross-tenant data access in negative tests;
2. zero stale-fence accepted effects;
3. zero duplicate semantic business effects in crash/redelivery suite;
4. zero deterministic-policy UNKNOWN or ERROR converted to PASS;
5. complete durable record for every accepted command and disposition;
6. 10x load test meets approved queue/latency and fairness targets;
7. Foundry route proof passes or fallback is selected;
8. restore and regional recovery drills meet approved RPO/RTO;
9. KYC golden cases pass through decision, QA and human review;
10. on-call, DLQ, reconciliation and cancellation runbooks are exercised.

20. Operational runbooks

Required runbooks:

- high READY/queue age;
- repeated lease expiry or stale fences;
- DLQ classification and controlled replay;
- model or tool provider outage;
- budget/quota exhaustion;
- plan, prompt, model, tool or skill revocation;
- policy service unavailable or stale policy cache;
- Context Service unavailable or unauthorized retrieval alert;
- unknown external side effect;
- PostgreSQL HA failover and connection recovery;
- Service Bus outage and dispatch reconstruction;
- regional activation and split-brain prevention;
- suspicious agent/tool behavior and execution-token revocation;
- audit/ledger sequence gap.

Every runbook defines trigger, severity, owner, immediate containment, diagnostic queries, safe recovery command, evidence to retain and closure criteria. Operators invoke versioned admin commands through the execution API; direct database modification is emergency-only, dual-controlled and followed by an immutable incident record.

21. Architecture decisions to close before production

Decision	Recommended baseline	Must be confirmed by
Agent runtime	Foundry hosted adapter if compatibility proof passes; Container Apps fallback otherwise	AI platform + security, week 2
Workflow authority	PostgreSQL WorkItems and transitions	Architecture review, week 1
Queue ordering	No sessions initially; add isolated session queue only for proven FIFO need	Runtime lead, week 4
Policy engine	OPA/Rego or equivalent pinned deterministic engine	Security/policy engineering, week 2
Database topology	Workflow schema colocated with Case server; control-plane registry separate	DBA + architecture, week 2
Multi-tenant isolation	Shared regulated stamp with composite keys/RLS; dedicated stamp option	Security + product, week 2
Regional pair	East US 2 / Central US proposal	Cloud platform + records, week 1
Recovery authority	Independently available stamp epoch/fencing mechanism	SRE + security, week 4
Model profiles/quotas	Regional U.S. deployments, no Global	AI risk + cloud platform, week 3
Effect authorization	Typed effect classes and approval/compensation matrix	KYC operations + security, week 4

22. Definition of done

The execution control plane is production-ready when a signed KYC plan can admit a case, construct dependent work, select only authorized and attested profiles, reserve budgets, build governed context, run through either supported runtime, mediate all models/tools through APIM, reject stale workers, accept only validated commands, route required human review, and recover from duplicate delivery or worker death without losing work or duplicating a semantic business effect.

It must also demonstrate that Service Bus can be rebuilt from PostgreSQL authority, a revoked dependency stops new affected work, an ambiguous external effect enters reconciliation, a changed Case invalidates stale decisions, and a regional recovery cannot produce two active writers. Operations must reconstruct every accepted outcome from the Case ledger, WorkItem transitions, ContextManifest, policy results and ExecutionRecord.

This keeps cognition replaceable while making lifecycle, authority, evidence and recovery deterministic.